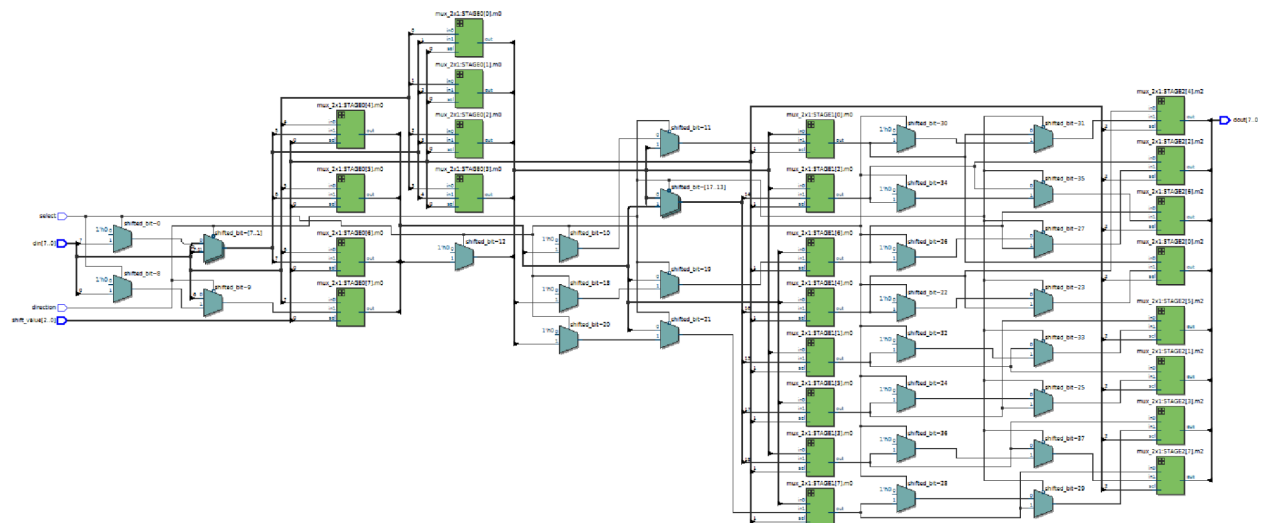
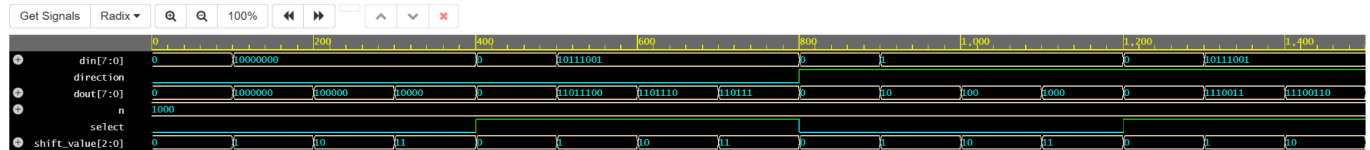




<<Filter>>

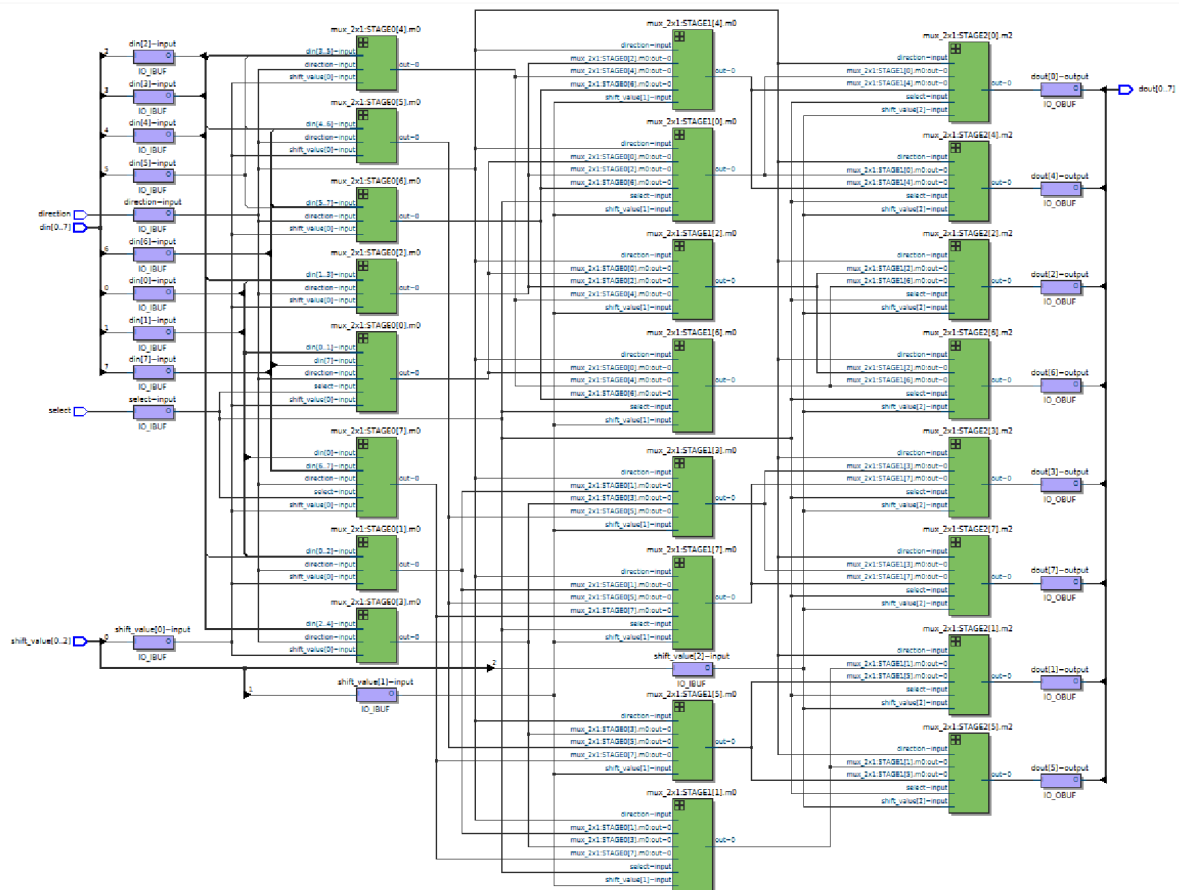
	Resource	Usage
	Estimate of Logic utilization (ALMs needed)	15
▼	Combinational ALUT usage for logic	24
	-- 7 input functions	0
	-- 6 input functions	6
	-- 5 input functions	18
	-- 4 input functions	0
	-- <=3 input functions	0
	Dedicated logic registers	0
	I/O pins	21
	Total DSP Blocks	0
0		
1	Maximum fan-out node	dir...put
2	Maximum fan-out	24
3	Total fan-out	155
4	Average fan-out	2.35



Note: To revert to EPWave opening in a new browser window, set that option on your profile page.

```
# run -all
# time=0, select=0 shift_value=000 din=00000000 dout=00000000
# time=100, select=0 shift_value=001 din=10000000 dout=01000000
# time=200, select=0 shift_value=010 din=10000000 dout=00100000
# time=300, select=0 shift_value=011 din=10000000 dout=00010000
# time=400, select=1 shift_value=000 din=00000000 dout=00000000
# time=500, select=1 shift_value=001 din=10111001 dout=11011100
# time=600, select=1 shift_value=010 din=10111001 dout=01101110
# time=700, select=1 shift_value=011 din=10111001 dout=00110111
# time=800, select=0 shift_value=000 din=00000000 dout=00000000
# time=900, select=0 shift_value=001 din=00000001 dout=00000010
# time=1000, select=0 shift_value=010 din=00000001 dout=00000100
# time=1100, select=0 shift_value=011 din=00000001 dout=00001000
# time=1200, select=1 shift_value=000 din=00000000 dout=00000000
# time=1300, select=1 shift_value=001 din=10111001 dout=01110011
# time=1400, select=1 shift_value=010 din=10111001 dout=11100110
# time=1500, select=1 shift_value=011 din=10111001 dout=11001101
# ** Note: $stop : testbench.sv(79)
# Time: 1600 ns Iteration: 0 Instance: /barrel_shifter_testbench
# Break in Module barrel_shifter_testbench at testbench.sv line 79
# Stopped at testbench.sv line 79
# exit
# End time: 18:47:47 on Feb 07,2026, Elapsed time: 0:00:01
# Errors: 0, Warnings: 0
```

I went with the 3-stage mux per bit design, but I left the change direction to some if statements that checked the direction instead of storing a separate string of bits based on inversion or not- it made more sense for me to be able to see what was happening when going left. I did look at an inversion method, and it makes sense not have to run a check on direction at each operation, but I don't need space for an extra variable the way I did it. As you can see above, the print statements given in the first testbench do a great job of showing that the output works as desired with the 1 rolling right and left for logical and the ones coming around for rotation. I also followed the specs and made it work with 8-bit but did not hard code 8 into the parameter, although I didn't implement a nested loop so it would actually work with larger numbers because I have a set number of stages, but I do know that all I would have to do is convert from the 3 separate stages to looping over the stages. Below is the post-mapping schematic, where you can clearly see the three separate stages needed for the 8-bit barrel shifter.



// 2to1 Multiplexor behavioral code

```
module mux_2x1(
    input in0, in1,
    input sel,
    output logic out
);
```

```
    assign out =sel? in1 : in0;
endmodule
```

```
module barrel_shifter #(
    parameter int n = 8 //declared to be variable but would need nested for loop for general case
    (just building for 8 for requirements)
)(
    input logic [$clog2(n)-1:0] shift_value,
    input logic direction,      // 0 = right, 1 = left
    input logic [n-1:0] din,
    input logic select,
```

```

output logic [n-1:0] dout

);

logic [n-1:0] stage0, stage1;
genvar i;
generate
for (i = 0; i < n; i++) begin : STAGE0 //using n but doesn't work for n>8 (mvp)
    logic shifted_bit;

    always_comb begin
        if (direction == 0) begin // right
            if (select) // rotate
                shifted_bit = din[(i+1) % n];
            else // logical
                shifted_bit = (i+1 < n) ? din[i+1] : 1'b0;
        end
        else begin // left
            if (select) // rotate
                shifted_bit = din[(i-1+n) % n];
            else // logical
                shifted_bit = (i-1 >= 0) ? din[i-1] : 1'b0;
        end
    end

    mux_2x1 m0 (
        .in0(din[i]),
        .in1(shifted_bit),
        .sel(shift_value[0]),
        .out(stage0[i])
    );
end
endgenerate

generate
for (i = 0; i < n; i++) begin : STAGE1
    logic shifted_bit;

    always_comb begin
        if (direction == 0) begin // right
            if (select) // rotate
                shifted_bit = stage0[(i+2) % n];
            else // logical
                shifted_bit = (i+2 < n) ? stage0[i+2] : 1'b0;
        end
        else begin // left
            if (select) // rotate
                shifted_bit = stage0[(i-2+n) % n];
            else // logical
                shifted_bit = (i-2 >= 0) ? stage0[i-2] : 1'b0;
        end
    end

    mux_2x1 m1 (
        .in0(stage0[i]),
        .in1(shifted_bit),
        .sel(shift_value[1]),
        .out(stage1[i])
    );
end
endgenerate

```

```

    end
    else begin // left
        if (select) // rotate
            shifted_bit = stage0[(i-2+n) % n];
        else // logical
            shifted_bit = (i-2 >= 0) ? stage0[i-2] : 1'b0;
        end
    end
end

mux_2x1 m0 (
    .in0(stage0[i]),
    .in1(shifted_bit),
    .sel(shift_value[1]),
    .out(stage1[i])
);
end
endgenerate

generate
for (i = 0; i < n; i++) begin : STAGE2
    logic shifted_bit;

    always_comb begin
        if (direction == 0) begin // right
            if (select) // rotate
                shifted_bit = stage1[(i+4) % n];
            else // logical
                shifted_bit = (i+4 < n) ? stage1[i+4] : 1'b0;
            end
        else begin // left
            if (select) // rotate
                shifted_bit = stage1[(i-4+n) % n];
            else // logical
                shifted_bit = (i-4 >= 0) ? stage1[i-4] : 1'b0;
            end
        end
    end

    mux_2x1 m2 (
        .in0(stage1[i]),
        .in1(shifted_bit),
        .sel(shift_value[2]),
        .out(dout[i])
    );
end
end

```

endgenerate

endmodule